SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192511053**
Name	**B Santhosh Kumar**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

*I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar
Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

Question 1: AI-Powered Drone Navigation in a Flood-Affected Region

Aim:

To analyze how Greedy Best-First Search and A* Search help an AI-powered drone deliver emergency medicines safely and quickly in a flood-affected region, and to recommend the most suitable algorithm for dynamic real-time navigation.

Introduction:

During floods, many roads become blocked unexpectedly and weather conditions can increase travel time. An AI-powered drone must quickly find a safe route to deliver emergency medicines while minimizing delay and avoiding risky areas. Since the environment changes continuously, the search algorithm should be able to use available information efficiently and adapt whenever new updates are received.

Problem Statement:

A government deploys AI-powered drones to deliver medicines in flood-affected areas. The drone has access to:

- Straight-line distance to the destination (heuristic)
- Partial real-time updates about blocked roads
- Variable movement costs caused by weather and terrain

The objective is to reach the destination with minimum delay while ensuring safe navigation.

Problem Formulation:

Component	**Description**
Initial State	Drone at the starting location
Goal State	Deliver medicines to the affected area
State Representation	Drone's current GPS location
Operators	Move to neighbouring safe locations
Search Algorithms	Greedy Best-First Search, A* Search
Heuristic	Straight-line distance to destination
Path Cost	Distance + weather + terrain cost

Block Diagram:

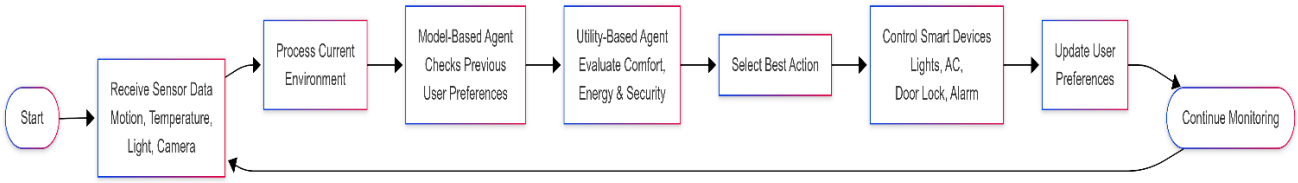

Figure 1: Workflow of Smart Home Automation System Using Model-Based and Utility-Based Intelligent Agents

Source: Author's illustration created using Mermaid Live Editor based on the given scenario.

Greedy Best-First Search:

Greedy Best-First Search always chooses the next location that appears closest to the destination according to the heuristic value. It ignores the cost already travelled and focuses only on reaching the goal quickly.

Working in this Scenario

- The drone checks nearby locations.
- It calculates the straight-line distance to the destination.
- The location with the smallest heuristic value is selected.
- This process continues until the medicines are delivered.

Strengths

- Makes decisions quickly.
- Requires less computation.
- Suitable when an immediate route is needed.

Weaknesses

- Does not consider actual travel cost.
- May choose unsafe or expensive routes.
- Can move toward blocked paths if the heuristic is misleading.

***A Search*:**

A* Search considers both the distance already travelled and the estimated distance to the destination.

It uses the evaluation function:

f(n) = g(n) + h(n)

where

- **g(n)** = actual travel cost

- **h(n)** = estimated distance to the destination

Working in this Scenario

- The drone calculates both the travelled cost and estimated remaining distance.
- Weather conditions and difficult terrain increase the movement cost.
- The path with the smallest total cost is selected.
- If a blocked path is detected, A* recalculates a better route.

Advantages

- Finds an optimal route when the heuristic is admissible.
- Considers both safety and travel cost.
- Produces more reliable routes than Greedy Search.

Impact of Heuristic Accuracy:

The heuristic function greatly affects the performance of both algorithms.

- If the heuristic is accurate, Greedy Search reaches the destination faster.
- If the heuristic is inaccurate, Greedy Search may select poor routes.
- A* is less affected because it also considers the actual travel cost.
- A better heuristic reduces unnecessary node expansion and improves search efficiency.

Effect of Dynamic Changes

Flood conditions change continuously, causing roads to become blocked.

These changes affect the search process by:

- Invalidating previously selected routes.
- Increasing travel costs because of bad weather.
- Requiring the drone to recalculate its path.
- Increasing computation whenever new information is received.

Among the two algorithms, A* adapts more effectively because it evaluates both path cost and heuristic information.

Recommended Algorithm:

A* Search is the most suitable algorithm for this scenario.

It provides a good balance between speed and optimality by considering both the actual movement cost and the estimated distance to the destination. Even when weather conditions or blocked routes change during the mission, A* can find a safer and more efficient path than

Greedy Best-First Search. Although Greedy Search is faster, it may select risky routes because it ignores travel cost.

Result:

The flood-affected drone navigation problem can be solved using both Greedy Best-First Search and A* Search. Greedy Search provides faster decisions but may choose unsafe routes because it depends only on the heuristic value. A* Search combines actual travel cost with heuristic information, making it more suitable for dynamic environments where safety, optimality, and real-time decision-making are equally important. Therefore, *A Search is the recommended approach** for emergency medicine delivery.

QUESTION 2

Traffic Signal Optimization Using Hill Climbing and Simulated Annealing

Aim:

To analyze how Hill Climbing and Simulated Annealing optimize traffic signal timings in a smart city and recommend the most suitable optimization technique.

Introduction:

Modern smart cities use Artificial Intelligence to manage traffic efficiently. Traffic signals must continuously adjust according to changing traffic flow to reduce waiting time, fuel consumption, and congestion. Since hundreds of intersections are connected, the search space becomes very large. Traditional search methods are inefficient because evaluating every possible signal timing requires **excessive** computation.

Problem Statement:

A smart city uses an AI system to optimize traffic signal timings at multiple intersections. The system must minimize waiting time, fuel consumption, and traffic congestion while adapting to continuously changing traffic conditions.

Problem Formulation:

Component	Description
Initial State	Current traffic signal timings
Goal State	Optimized traffic flow
State Representation	Signal timing configuration
Operators	Increase or decrease green signal duration
Objective	Minimize waiting time, fuel usage, and congestion

Hill Climbing:

Hill Climbing improves the current solution by selecting a neighboring solution with a better evaluation value.

Working

- Start with current signal timings.
- Generate nearby timing combinations.
- Compare traffic performance.

- Select the better configuration.
- Repeat until no further improvement is possible.

Advantages

- Simple to implement.
- Fast convergence.
- Suitable for local optimization.

Limitations

- May stop at local maxima.
- Cannot handle plateaus efficiently.
- May miss the global optimum.

Local Search Problems:

Local Maxima

The algorithm reaches a better solution than its neighbors but not the best overall solution.

Example: Traffic improves at one junction while congestion remains high in nearby intersections.

Plateau

Several neighboring solutions have the same evaluation value.

Example: Different signal timings produce similar traffic flow.

Ridge

The best solution requires moving through less favorable states before improving.

Example: Small timing changes do not improve traffic, but a larger adjustment would.

Simulated Annealing:

Simulated Annealing sometimes accepts poorer solutions during the search. This helps escape local maxima and increases the chance of finding a better overall solution.

Advantages

- Escapes local maxima.
- Explores larger search space.
- Produces better global solutions.

Limitations

- Requires temperature tuning.
- Takes more computation time.

Recommended Solution:

Simulated Annealing is more suitable because traffic conditions change continuously. It explores multiple possible solutions and avoids getting trapped in locally optimal signal timings, making it more scalable for large smart city networks.

Block Diagram

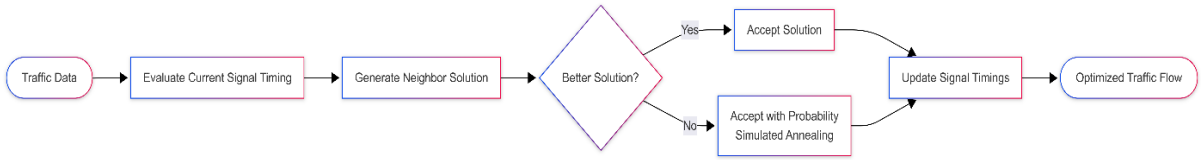

Figure 1: Traffic Signal Optimization Using Simulated Annealing

Source: Author's illustration created using Mermaid Live Editor based on the given scenario.

Result:

The traffic optimization problem is effectively solved using Simulated Annealing because it adapts to changing traffic conditions, avoids local optima, and produces better overall traffic management than Hill Climbing.

QUESTION 3

Autonomous Mars Rover Using Online Search Agent

Aim:

To analyze how an online search agent enables an autonomous Mars rover to safely explore unknown terrain and collect samples.

Introduction:

Mars exploration involves navigating an unknown environment with limited sensing capability. Since the rover has no complete map, it must observe nearby locations, avoid hazards, and update its knowledge while moving. Online Search is suitable because decisions are made during exploration instead of before the mission begins.

Problem Statement:

An autonomous rover explores the Martian surface to collect samples while avoiding rocks and craters. The rover operates with limited sensing capability and continuously updates its knowledge during navigation.

Problem Formulation:

Component	Description
Initial State	Rover at starting position
Goal State	Reach sample collection point safely
State Representation	Rover position
Operators	Move Up, Down, Left, Right
Search Strategy	Online Search Agent
Objective	Safe navigation with minimum risk

Why Online Search?

The rover cannot use Offline Search because the terrain is unknown before exploration. It must make decisions using only the information collected during movement.

Challenges

- Limited sensing capability.
- Unknown obstacles.
- Dynamic terrain conditions.
- No complete map.
- Communication delay with Earth.

Internal Model Update:

The rover continuously updates its internal map by

- Detecting nearby obstacles.
- Marking safe paths.
- Recording explored locations.
- Selecting the next safest movement.

Online Search vs Other Search Methods:

Search Method	Characteristics
Uninformed Search	Explores without additional knowledge
Informed Search	Uses heuristic information
Online Search	Learns while exploring unknown environments

Recommended Solution:

An Online Search Agent is the best choice because it continuously updates its knowledge and adapts to unexpected hazards. This approach improves navigation safety and supports real-time decision making.

Block Diagram:

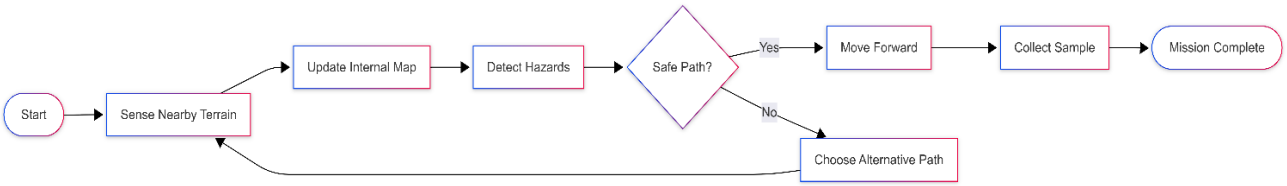

Figure 1: Autonomous Mars Rover Using an Online Search Agent

Source: Author's illustration created using Mermaid Live Editor based on the given scenario.

Result:

The Online Search Agent enables the Mars rover to safely explore unknown terrain by continuously sensing the environment, updating its internal model, avoiding hazards, and selecting appropriate actions. This approach provides reliable and adaptive navigation under uncertain conditions.

QUESTION 4

AI-Based University Exam Timetable Generation Using Constraint Satisfaction Problem (CSP)

Aim:

To formulate the university exam timetabling problem as a Constraint Satisfaction Problem (CSP) and analyze how backtracking and constraint propagation generate a valid timetable efficiently.

Introduction:

Preparing an examination timetable is a complex scheduling problem because multiple constraints must be satisfied simultaneously. The AI system should assign exams to available time slots and halls without causing conflicts among students, faculty, or resources. Constraint Satisfaction Problems (CSPs) provide an effective way to solve such scheduling tasks.

Problem Statement:

A university needs to generate an examination timetable while satisfying constraints such as student conflicts, hall availability, faculty availability, subject order, and special accommodations.

CSP Formulation:

Variables

Each subject to be scheduled.

Example:

- Mathematics
- Physics
- Chemistry
- Programming

Domains

Possible examination time slots and available halls.

Example:

- Morning
- Afternoon
- Hall A
- Hall B

Constraints:

- No student should have overlapping exams.
- Only one exam can be conducted in a hall at a time.
- Faculty must be available during the assigned session.
- Certain subjects must be conducted before others.
- Students requiring special accommodations should receive suitable halls.

Backtracking Search:

Backtracking assigns exams one by one.

If a constraint is violated, the current assignment is removed and another available option is tried until a valid timetable is obtained.

Advantages

- Produces valid schedules.
- Easy to implement.
- Suitable for CSP problems.

Limitation

- Search time increases as the number of courses grows.

Forward Checking (Constraint Propagation):

Forward Checking improves efficiency by removing invalid choices immediately after every assignment.

For example, if Hall A is assigned during the Morning session, that option is removed from the remaining subjects.

Benefits

- Reduces unnecessary search.
- Detects conflicts early.
- Speeds up timetable generation.

Real-World Challenges:

- Large number of courses.
- Limited examination halls.
- Faculty availability changes.
- Student elective conflicts.
- Special seating arrangements.

Recommended Solution:

Backtracking combined with Forward Checking is the most suitable approach. Backtracking guarantees a valid timetable, while Forward Checking reduces the search space and improves performance for large universities.

Block Diagram:

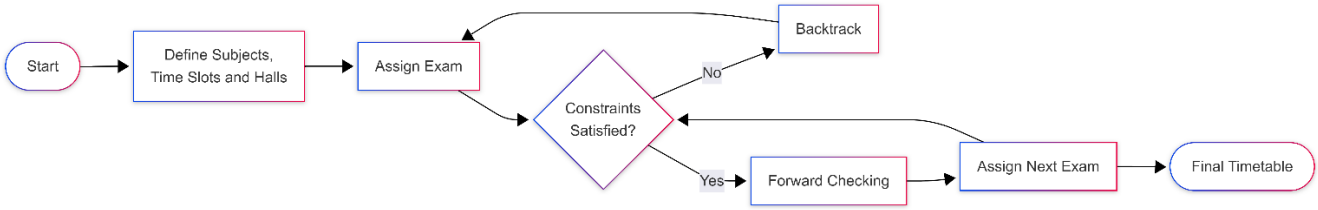

Figure 1: AI-Based Exam Timetable Generation Using CSP

Source: Author's illustration created using Mermaid Live Editor based on the given scenario.

Result:

The examination timetable can be generated efficiently using Constraint Satisfaction Problems. Backtracking ensures a valid schedule, while Forward Checking improves efficiency by eliminating invalid assignments during the search process.

QUESTION 5

Strategic Game AI Using Minimax and Alpha-Beta Pruning

Aim:

To analyze how Minimax and Alpha-Beta Pruning enable an AI player to make efficient decisions in a real-time strategy game.

Introduction:

Modern strategy games require AI opponents to make intelligent decisions within a short time. Since every move generates many possible future game states, the decision tree becomes extremely large. Minimax and Alpha-Beta Pruning help reduce computation while selecting strong moves.

Problem Statement:

An AI opponent must compete against human players by evaluating multiple game states and selecting the best possible move within limited time.

Minimax Algorithm:

Minimax assumes both players make optimal decisions.

- The AI tries to maximize its score.
- The opponent tries to minimize the AI's score.
- The algorithm evaluates future game states and selects the move with the best outcome.

Advantages

- Produces strong decisions.
- Suitable for two-player games.
- Guarantees optimal play when the full tree is explored.

Limitation

- Computationally expensive for large search trees.

Computational Challenges:

- Large branching factor.
- Huge game tree.
- High memory usage.
- Real-time decision deadlines.

Alpha-Beta Pruning:

Alpha-Beta Pruning reduces unnecessary node evaluation without changing the final decision. It ignores branches that cannot produce a better result than the current best move.

Advantages

- Reduces computation.
- Explores deeper game states.
- Produces the same result as Minimax with fewer evaluations.

Evaluation Function:

The AI evaluates each game state using factors such as

- Player score
- Remaining resources
- Army strength
- Health
- Territory control
- Distance to enemy
- Defensive position

Higher evaluation values indicate better positions for the AI.

Recommended Solution:

Minimax combined with Alpha-Beta Pruning is the most suitable solution. Minimax selects the best move, while Alpha-Beta Pruning significantly reduces computation, allowing the AI to respond quickly during gameplay.

Block Diagram:

Figure 1: Strategic Game Decision Making Using Minimax and Alpha-Beta Pruning

Source: Author's illustration created using Mermaid Live Editor based on the given scenario.

Result:

Minimax and Alpha-Beta Pruning provide an effective solution for strategic game AI. Minimax identifies the best move, while Alpha-Beta Pruning reduces unnecessary computation, enabling faster and more efficient real-time decision making.